

Hackthon 1 preparation - solutions

1. REPRESENT

1.1 `nadia[enrolledIn -> course_ai]`. Reversed: ``course_ai[enrolledIn -> nadia]` claims *the course* is enrolled in Nadia — backwards and meaningless under this schema. Not the same fact: F-Logic facts are directional ( `subject[attribute -> object]`), so swapping subject and object produces a different claim, not an equivalent restatement of the first. **A fact is a claim with a direction!**

Concept note. Students coming from math or spreadsheets tend to read `a[r -> b]` as a symmetric relationship, the way `a = b` is symmetric. It isn't! it's a sentence with a subject and an object, closer to natural language ("Isaac's father is Abraham") than to an equation.

1.2 Three bindings, one per matching fact:

```
S = nadia, C = course_ai
S = tom,   C = course_ai
S = nadia, C = course_logic
```

Each variable stands for whatever value satisfies its position in one matching fact. One row per match, not one row per distinct name.

Concept note. The exercise is designed so `nadia` appears twice. If you think of a variable like a spreadsheet lookup ("give me Nadia's course") you expect one row per student. But a query is asking for every combination that matches, not a deduplicated summary.

1.3 Join variable: `?C`.

```
?- ?S[enrolledIn -> ?C] AND ?C[taughtBy -> hoffmann].
```

Result: `S = nadia, C = course_ai` and `S = tom, C = course_ai`. Nadia's `course_logic` enrollment is excluded, nothing says who teaches it.

Concept note. "a shared variable is the whole join. Nothing else connects two facts."

1.4 Renaming one `?C` to `?C2` removes the shared variable, so the two conditions no longer constrain each other. The query becomes an unconstrained combination: every `(S, C)` from the first condition paired with every course from the second, regardless of whether it's the *same* course. This is the direct proof that "join = shared variable". Remove the sharing and you get a cross-product, not a filtered join.

2. DERIVE

2.1

```
?S:AISStudent <- ?S[enrolledIn -> course_ai].
```

```
?- ?S:AISStudent. → nadia, tom.
```

Concept note. This is the baseline "head if body" pattern from slide 13: **declarative vs. imperative**. You are not writing a function that computes `AISStudent` when called; you are stating a standing fact-generation rule that holds for the entire life of the program. Every time a new `enrolledIn -> course_ai` fact is added anywhere in the base, this class membership follows automatically.

2.2

```
?A[requiredBefore -> ?B] <- ?A[prerequisiteOf -> ?B].  
  ?A[requiredBefore -> ?C] <- ?A[prerequisiteOf -> ?B] AND ?B[requiredBefore -> ?C].
```

One rule can't do it alone because its body only ever looks at *base* facts. It can capture a direct prerequisite but has no way to reach two hops without a second rule per additional hop. The recursive rule instead reuses its own previously-derived conclusions (`?B[requiredBefore -> ?C]`) to extend the chain by exactly one hop per round, so a fixed pair of rules covers a chain of any length.

2.3

Round	New <code>requiredBefore</code> facts
1	(stats101, ml201), (ml201, nlp301)
2	(stats101, nlp301)
3	none — fixpoint

(stats_101, nlp_301) cannot appear in round 1, because the second rule needs `ml_201[requiredBefore -> nlp_301]` as a premise, and that fact doesn't exist yet until round 1 has finished.

2.4 Deleting `stats_101[prerequisiteOf -> ml_201]` leaves only `ml_201[prerequisiteOf -> nlp_301]` as a base fact. Surviving derived pair: (ml_201, nlp_301). Lost: (stats_101, ml_201) and (stats_101, nlp_301). Removing a *derived* fact directly would not be meaningful: derived facts aren't stored as independent state, they're recomputed from current base facts every time the rules run. "Retracting" one changes nothing as long as its supporting base facts remain, only changing a base fact or a rule changes what's derivable.

2.5 Example: `stats_101[requiredBefore -> nlp_301]` rests on `stats_101[prerequisiteOf -> ml_201]` and `ml_201[requiredBefore -> nlp_301]`, which in turn rests on `ml_201[prerequisiteOf -> nlp_301]` — the same shape as slide 18's `joseph[ancestor -> abraham]` chain.

3. MODEL

3.1 Two handles, e.g. `course_ai_fall126` and `course_ai_spring27`, each with `label -> "Introduction to AI"`. The wrong fact would be using **one** handle for both offerings — e.g. writing all fall and spring enrollment/teaching facts onto a single `course_ai` object. That would incorrectly claim there is only one offering, so a student enrolled in the fall section would appear enrolled in the spring section too, and the two sections' distinct teachers/schedules would collide into one record.

3.2 Model it as its own entity (a `Submission`), by the same argument as slide 25's "a person is not a birth record": the late-submission facts (timestamp, penalty) describe a specific *event*, not a permanent property of Nadia. Bolting `submittedLate -> true` directly onto `nadia` only allows one such fact ever (a functional-slot collision across multiple assignments), can't carry a timestamp or penalty of its own, and risks mixing up "correcting one submission record" with "erasing facts about the student."

3.3

```
Submission {  
    ofStudent: Student,  
    assignment: String,  
    submittedAt: String,  
    penalty: Integer  
}.  
  
sub_hw3_nadia:Submission.  
sub_hw3_nadia[ofStudent -> nadia; assignment -> "hw3";  
               submittedAt -> "2026-09-20T23:59:00"; penalty -> 25].
```

```
?- ?S:Submission AND ?S[penalty -> ?P] AND ?P > 0.
```

3.4

```

declare required Submission ofStudent.

declare functional ofStudent.

?S:HeavyPenalty <- ?S:Submission AND ?S[penalty -> ?P] AND ?P > 20.

Enrollment { ofStudent: Student, course: String, grade: Integer }.
e1:Enrollment.
e1[ofStudent -> nadia; course -> "ai"; grade -> 88].

?St[gradeFor(?Course) -> ?G] <-
    ?E:Enrollment AND ?E[ofStudent -> ?St] AND ?E[course -> ?Course] AND ?E[grade ->
?G].

?- nadia[gradeFor("ai") -> ?G].    // G = 88

```

4. DECIDE

This section is the one most worth spending live time on. Everything before it (REPRESENT, DERIVE, MODEL) is about getting a *true* answer. DECIDE is about noticing when the system is right to refuse one — and that instinct runs against most students' prior programming experience, where a missing value usually crashes, defaults to zero, or is silently falsy. Here, "we don't know" is a first-class, correct answer, and treating it as `false` is the single most consequential mistake a hackathon team can make in their final demo.

4.1 `?- tom[submitted -> hw1].` → `unknown` (no evidence either way). `?- NOT tom[submitted -> hw1].` → also `unknown` — under open world, `NOT` only succeeds when the positive fact can be *proven false*, not merely when it's absent. Since nothing here proves Tom didn't submit, negating an unknown stays unknown. These are not opposites: `unknown` and `NOT-provable` are the same non-answer here, not a true/false pair.

4.2 `closed class withdrawn.` is justified: withdrawal is a tracked administrative action with a complete list. The rule `?S:activelyEnrolled <- ?S:student AND NOT ?S:withdrawn.` is therefore sound. `NOT ?S:withdrawn` correctly reads "not on the complete withdrawal list" as true. The equivalent move over `submitted` (`?S:missingHW1 <- ?S:student AND NOT ?S[submitted -> hw1].`) is **not** justified, because the submission list is only partial: absence of a submission record doesn't mean the student didn't submit, only that it isn't recorded yet. Closing that slot would convert "not yet entered" into a confident (and possibly wrong) "false".

4.3 The rule itself is not wrong under open-world semantics: for a student with no `hasGrade` fact, the rule body has nothing to match, so `?S:passing` correctly comes back `unknown`. The actual danger is downstream: whoever *reads* the query result must not silently treat "no `:passing` answer" as "failing."

Doing so converts missing data (an ungraded student) into a false negative (marked as failed) without any evidence for it, the same category of mistake as the lecture's wedding rule, just from conflating "missing" with "false" instead of "some" with "all."

4.4 Evaluate with the two-level truth table from slide 33 (OR within a group: true if any alternative true, false if all false, else unknown; AND across groups: true if all groups true, false if any group false, else unknown):

Student	Group 1 (attendance OR waiver)	Group 2 (final submitted)	Eligible?
A	true (≥ 8 sessions alone is enough)	true	true
B	false (both alternatives false)	true	false (one group is false)
C	true (≥ 8 sessions alone is enough, waiver irrelevant)	unknown	unknown (no group is false, but not all are true)

4.5 For student C, the fact causing **unknown** is specifically whether the final project was submitted, group 1 is already settled **true** because the attendance count alone clears the bar, independent of the waiver. If "submitted final" had been wrongly closed (no record treated as "did not submit"), C would have been marked **ineligible** outright, potentially denying a certificate to someone who did submit but whose submission simply wasn't logged yet.